

Inventory Management System

A structured, phase-by-phase execution roadmap covering database foundation, standardized development workflow, and six functional modules — built on ASP.NET Core MVC with a Code-First approach.

TRACK

ITI .NET Full Stack

ARCHITECTURE

ASP.NET Core MVC

APPROACH

Code-First (EF Core)

MODULES

6 Core Modules

MODULE SCOPE

Products & Categories

Suppliers & Vendors

Purchases (Stock In)

Sales (Stock Out)

Dashboard & Analytics

Generative AI

Table of Contents

Foundation, database, workflow, six modules, and a requirements check.

PHASE 1 — 2 · SETUP, DATABASE & WORKFLOW

01	Foundation & Database Setup	03
	ERD, entity classes, ApplicationDbContext, migrations & deployment	
1A	Database Schema Reference	04
	All 8 tables: exact table, class, column, and key names	
1B	Relationships, Constraints & Naming	05
	Foreign keys, delete rules, indexes, precision, naming rules	
02	Standardized Module Workflow	06
	The 4-step sequence every developer follows per module	

PHASE 3 · DETAILED MODULE BREAKDOWN

M1	Products & Categories Management	07
	Catalog CRUD, category view, filtering, pagination, SKU generation	
M2	Suppliers & Vendor Mapping	08
	Supplier profiles, product mapping, contract pricing	
M3	Purchases & Stock Replenishment	09
	Procurement orders and automatic stock addition	
M4	Sales & Inventory Deduction	10
	POS transactions and automatic stock deduction	
M5	Dashboard & Analytics	11
	KPIs, low-stock alerts, most-sold products, activity log	
M6	Generative AI Integration	12
	Chatbot, RAG, natural-language analytics, restock insights, reports	

APPENDIX

A	Requirements Coverage	13
	Every requirement from the specification mapped to the module that delivers it	

Foundation & Database Setup

Core infrastructure and database configuration required before any module development begins.

1

ERD & DATABASE SCHEMA

Complete ERD with **8 core tables**, named exactly as in the requirements:

Categories

Suppliers

Products

SupplierProducts

Purchases

PurchaseItems

Sales

SalesItems

AIChatLogs (optional)

SupplierProducts is the junction table for the Suppliers ↔ Products many-to-many relationship. AIChatLogs is outside the required schema. Column details: page 04.

2

ENTITIES & DBCONTEXT

- ▶ Build one **C# entity class per table** (singular: Category, Supplier, Product, SupplierProduct, Purchase, PurchaseItem, Sale, SaleItem) with **PK/FK property names exactly as in the spec** (ProductID, SupplierProductID, ...).
- ▶ Add **navigation properties** on both sides of every relationship.
- ▶ Configure **ApplicationDbContext** with one DbSet per table and the **Fluent API**.
- ▶ Enforce **foreign keys**, delete behaviors, a **unique composite index** on SupplierProducts (SupplierID, ProductID), and decimal precision (see page 05).

3

MIGRATIONS & DEPLOYMENT

- ▶ Run **Add-Migration InitialCreate**.
- ▶ Run **Update-Database** to generate the SQL database, then verify table and column names against the schema reference.
- ▶ Push the foundational codebase to **main** on GitHub for the team.

NEXT · PHASE 01

Database Schema Reference



Database Schema Reference

Table, class, and column names match the specification exactly. C# types and lengths are suggested.

Categories

class Category

CategoryID	int	PK
CategoryName	string(100)	
Description	string(500)?	

Suppliers

class Supplier

SupplierID	int	PK
SupplierName	string(150)	
ContactName	string(100)	
Phone	string(30)	
Email	string(150)	
Address	string(250)	

Products

class Product

ProductID	int	PK
SKU	string(50)	UQ
ProductName	string(200)	
CategoryID	int	FK
UnitPrice	decimal(18,2)	
StockQuantity	int	
LowStockThreshold	int	
UNIQUE (SKU)		

SupplierProducts

class SupplierProduct

SupplierProductID	int	PK
SupplierID	int	FK
ProductID	int	FK
SupplierSKU	string(50)	
ContractPrice	decimal(18,2)	
LeadTimeDays	int	
UNIQUE (SupplierID, ProductID)		

Purchases

class Purchase

PurchaseID	int	PK
SupplierID	int	FK
PurchaseDate	DateTime	
TotalAmount	decimal(18,2)	

PurchaseItems

class PurchaseItem

PurchaseItemID	int	PK
PurchaseID	int	FK
ProductID	int	FK
Quantity	int	
UnitCost	decimal(18,2)	

Sales

class Sale

SaleID	int	PK
SaleDate	DateTime	
TotalAmount	decimal(18,2)	
CustomerInfo	string(200)?	

SalesItems

class SaleItem

SaleItemID	int	PK
SaleID	int	FK
ProductID	int	FK
Quantity	int	
UnitPrice	decimal(18,2)	

PK primary key FK foreign key UQ unique ? = nullable

Relationships, Constraints & Naming

Configure these in the Fluent API inside `ApplicationDbContext` before running the first migration.

RELATIONSHIP (1 : N)	FOREIGN KEY	ON DELETE
Categories → Products	Products.CategoryID	Restrict
Suppliers → Purchases	Purchases.SupplierID	Restrict
Suppliers → SupplierProducts	SupplierProducts.SupplierID	Cascade
Products → SupplierProducts	SupplierProducts.ProductID	Cascade
Purchases → PurchaseItems	PurchaseItems.PurchaseID	Cascade
Products → PurchaseItems	PurchaseItems.ProductID	Restrict
Sales → SalesItems	SalesItems.SaleID	Cascade
Products → SalesItems	SalesItems.ProductID	Restrict

Suppliers ↔ Products is Many-to-Many through SupplierProducts (the two SupplierProducts rows above).

CONSTRAINTS & FLUENT API

- **Unique composite index** on SupplierProducts (SupplierID, ProductID). Primary keys stay single-column, as in the spec.
- **Unique index** on Products.SKU; required: CategoryName, ProductName, SKU, SupplierName.
- **Precision** decimal(18,2) on UnitPrice, ContractPrice, UnitCost, and both TotalAmount columns.
- **Check constraints**: StockQuantity ≥ 0, LowStockThreshold ≥ 0, Quantity > 0.
- **Restrict** protects history: a category, supplier, or product with related records cannot be deleted; show a friendly message.

NAMING RULES

- PK and FK names follow **<Entity>ID** exactly as written in the spec.
- Entity classes are **singular**; tables and DbSet's are **plural**. **SalesItems** keeps the spec's table name; its class is **SaleItem**.
- **Navigation properties**: a collection on the parent (Category.Products) and a reference on the child (Product.Category).
- **Product status** (In Stock / Low Stock / Out of Stock) is computed, not stored.
- Extras outside the spec (AIChatLogs, ImagePath, IsActive) are optional and come last.

NEXT · PHASE 02

Standardized Module Workflow



Standardized Module Workflow

After pulling the latest updates from **main** into a dedicated feature branch, each developer follows this structured sequence to implement their assigned module.

1

Models / ViewModels

Build data contracts, request payloads, and view models.

2

Business Logic / Service Layer

Implement business rules, LINQ queries, and calculations.

3

Controllers / APIs

Build controller actions and endpoints connecting UI with services.

4

Views / UI

Create and style Razor Views for user interaction.

WHY THIS ORDER

Data contracts first ensures every layer above them has a stable shape to build against — logic, then controllers, then UI — minimizing rework as each module comes together.

[NEXT · MODULE 01](#)[Products & Categories Management](#)

Products & Categories Management

Entities: Category · Product

1

MODELS & VIEWMODELS

ProductViewModel

ProductDetailsViewModel

ProductCreateModel

ProductEditModel

ProductFilterModel

CategoryViewModel

CategoryCreateModel

CategoryDetailsViewModel

PagedResult<T>

2

BUSINESS LOGIC & SERVICES

- ▶ Full **CRUD** for products and categories: add, edit, delete, and view details.
- ▶ **Assign products to categories** with a required **CategoryID** dropdown on Create/Edit; the **category view** lists the products inside a category.
- ▶ Search and dynamic filtering by **name**, **category**, or **status** (computed from StockQuantity vs LowStockThreshold: In Stock, Low Stock, Out of Stock).
- ▶ Server-side **pagination** (Skip/Take, fixed page size) returning items plus total count.
- ▶ Unique **SKU** generation on create; block deleting a category that has products, or a product with purchase/sale history.

3

CONTROLLERS

ProductsController

CategoriesController

4

VIEWS / UI

- ▶ Products: paged List table (search box, category and status filters), Create, Edit, Details, Delete.
- ▶ Categories: List, Create, Edit, Delete, and a Details view showing the products in that category.

NEXT · MODULE 02

Suppliers & Vendor Mapping

Suppliers & Vendor Mapping

Entities: Supplier · SupplierProduct

1

MODELS & VIEWMODELS

SupplierViewModel

SupplierCreateModel

SupplierDetailsViewModel

SupplierProductMappingModel

2

BUSINESS LOGIC & SERVICES

- ▶ Supplier profiles: **SupplierName**, **ContactName**, **Phone**, **Email**, **Address** with full add / edit / delete / details.
- ▶ **Many-to-Many** mapping between Suppliers and Products through **SupplierProducts**: view the products each vendor supplies; add, edit, and remove mappings.
- ▶ Per-mapping data: **SupplierSKU**, **ContractPrice**, and **LeadTimeDays**.
- ▶ Prevent duplicate mappings (unique **SupplierID + ProductID**); block deleting a supplier that has purchases.

3

CONTROLLERS

SuppliersController

SupplierProductsController

4

VIEWS / UI

- ▶ Supplier Directory: searchable list with Create, Edit, Delete.
- ▶ Supplier Details: contact information plus a table of the products supplied.
- ▶ Vendor Product Mapping interface: pick a product, enter SupplierSKU, ContractPrice, LeadTimeDays.

NEXT · MODULE 03

Purchases & Stock Replenishment

Purchases & Stock Replenishment

Entities: Purchase · PurchaseItem · Supplier · Product · a.k.a. "Stock In"

1

MODELS & VIEWMODELS

PurchaseCreateModel

PurchaseItemModel

PurchaseReceiptViewModel

PurchaseListViewModel

2

BUSINESS LOGIC & SERVICES

- ▶ Create a purchase: select the **supplier**, set **PurchaseDate**, add line items (**ProductID**, **Quantity**, **UnitCost**).
- ▶ Product picker limited to the supplier's mapped products (SupplierProducts); **UnitCost** defaults to **ContractPrice**.
- ▶ **TotalAmount** = sum of Quantity × UnitCost, calculated on the server.
- ▶ Validation: at least one line item, Quantity > 0, no duplicate product lines.



AUTO STOCK ADDITION

On save, increase **StockQuantity** for every purchased product, inside the same database transaction as the Purchase and PurchaseItems inserts.

3

CONTROLLERS

PurchasesController

4

VIEWS / UI

- ▶ Purchase Order form: dynamic multi-item rows (add / remove) with a live total.
- ▶ Purchase Details receipt and Purchase history list.

NEXT · MODULE 04

Sales & Inventory Deduction

Sales & Inventory Deduction

Entities: Sale · SaleItem · Product · a.k.a. "Stock Out"

1

MODELS & VIEWMODELS

SaleCreateModel

SaleItemModel

SaleInvoiceViewModel

SalesSummaryViewModel

2

BUSINESS LOGIC & SERVICES

- ▶ Create a sale: **SaleDate**, **CustomerInfo**, and line items (**ProductID**, **Quantity**, **UnitPrice**).
- ▶ **UnitPrice** defaults to the product's current UnitPrice and is stored on the item as a price snapshot.
- ▶ **TotalAmount** = sum of Quantity × UnitPrice, calculated on the server.
- ▶ Validation: at least one item, Quantity > 0, and Quantity ≤ available **StockQuantity** (stock never goes negative).



AUTO STOCK DEDUCTION

On save, check stock and decrease **StockQuantity** for every sold product, inside the same database transaction as the Sale and SalesItems inserts.

3

CONTROLLERS

SalesController

4

VIEWS / UI

- ▶ Sales Invoice / POS screen: dynamic multi-item rows with a live total.
- ▶ Sale Details receipt and Sales history list.

NEXT · MODULE 05

Dashboard & Analytics

Dashboard & Analytics

Entities: Aggregates data across all system tables

1

MODELS & VIEWMODELS

DashboardStatsViewModel

LowStockAlertModel

TopSellingProductModel

RecentActivityModel

2

BUSINESS LOGIC & SERVICES

- ▶ Totals: **Products**, **Categories**, **Suppliers**, and total **StockQuantity**.
- ▶ **Purchases**: count and total value. **Sales**: count and total revenue.
- ▶ **Low-stock** products: $\text{StockQuantity} \leq \text{LowStockThreshold}$ (one shared rule reused by product status, dashboard, and AI).
- ▶ **Most sold products** ranked by total quantity sold and revenue (SalesItems grouped by product).
- ▶ **Recent activity** feed: latest sales and purchases merged and sorted by date.

3

CONTROLLERS

DashboardController

AnalyticsApiController (JSON for charts)

4

VIEWS / UI

- ▶ Centralized Dashboard: KPI cards, analytical charts (e.g. sales vs purchases, top products), low-stock alert table, most-sold table, and recent activity log.

NEXT · MODULE 06

Generative AI Integration

Generative AI Integration

Entities: AIChatLog (optional, not part of the required schema)

1

MODELS & VIEWMODELS

AIChatRequestModel

AIChatResponseModel

RestockRecommendationModel

AIReportModel

2

BUSINESS LOGIC & SERVICES

- ▶ **Inventory chatbot:** natural-language questions about products, stock, suppliers, purchases, and sales, using an LLM API (e.g., OpenAI, Gemini).
- ▶ **RAG:** retrieve only the relevant rows with LINQ and send them as grounded context to the LLM. Keep API keys in user-secrets.
- ▶ **Natural-language analytics:** "What are the most sold products this month?" and "Which products are currently low in stock?"
- ▶ **Restock recommendations** (from StockQuantity, LowStockThreshold, sales history) and **AI report generation** (summaries and insights from sales, purchases, and inventory).

! SCOPE

At least one meaningful AI feature is required: deliver the chatbot with RAG first.
Optional: a document assistant over uploaded documents.

3

CONTROLLERS

AIController

ChatbotController

4

VIEWS / UI

- ▶ Floating AI chatbot widget on every page, or a dedicated AI Assistant page.
- ▶ Restock recommendations and AI report panel (Dashboard or separate page).

APPENDIX

Requirements Coverage



Requirements Coverage

Each requirement group from the specification and where the plan delivers it.

1	Product Management	Add · Edit · Delete · View details · Browse in dynamic lists · Pagination · Search & filter by name, category, status	M1
2	Category Management	Add, edit, delete · Assign products to categories · View the products in a category	M1
3	Supplier Management	Supplier profiles · Contact info (phone, email, address) · Supplier ↔ Product mapping (Many-to-Many)	M2
4	Inventory Management	Real-time stock tracking · Auto increase on purchase · Auto decrease on sale · Low-stock alerts	M1 M3 M4 M5
5	Purchase Management	Create purchase · Supplier selection · Line items · Automatic total · Purchase details and history	M3
6	Sales Management	Create sale · Add sold products · Automatic total · Automated stock deduction · Sales receipts and history	M4
7	Dashboard & Statistics	Total products, categories, suppliers, stock · Low-stock count · Purchases and sales (count + value) · Recent activity · Most sold products	M5
8	Database Design	8 tables · exact table, class, PK/FK names · SupplierProducts junction · relationships and constraints	01 1A 1B
9	Generative AI	Chatbot · RAG · Natural-language analytics · Recommendations · Report generation · Document assistant (optional)	M6

DESIGN DECISIONS

- ▶ **Status filter** is computed from StockQuantity vs LowStockThreshold, so the schema matches the spec exactly.
- ▶ **Low-stock rule:** $\text{StockQuantity} \leq \text{LowStockThreshold}$, defined once and reused by products, dashboard, and AI.
- ▶ **Optional extras** (outside the spec): AIChatLogs, ImagePath, IsActive soft delete, AI document upload.

END OF PLAN

[Back to table of contents](#)

